

University of Texas at Dallas
Department of Electrical Engineering
EEDG/CE 6306 - Application Specific Integrated Circuit Design
Homework #3

Submission:

(a) Written Report:

- **MSDAP specification (20pts)**
- **Algorithm implementation description (10pts)**
- **Explanation of data3.out and data3_ref.out (5pts)**

(b) Your C/C++/Python source code (40pts)

(c) Your output file data2.out (15pts)

(d) Files generated by your HW#2 golden reference (data3.in, MSDAP_Coeff3.in, Rj3.in, data3_reference.out) and resulting output file from your MSDAP code (data3.out) (10pts)

Related input files are posted on eLearning, which include

(a) data1.in, MSDAP_Coeff1.in, Rj1.in, data1.out

(b) data2.in, MSDAP_Coeff2.in, Rj2.in

Submit onto eLearning a zip file (team): <First name #1>_<Last name #1>_<First name #2>_<Last name #2>_HW03.zip

* If you have any question about the homework, you may contact the TA

Purpose: Using HLL (C/C++/Python) develop a reference model to compare your verilog code against. We will also compare this model to the golden reference model built in HW#2. This is the first step in the MSDAP ASIC development.

Write a **C/C++/Python** program to accomplish the computation presented in the MSDAP paper. A **virtual address coefficient** and **rj** is introduced to do the

computation. Please carefully read the following example and data format descriptions.

Convolution Example Review:

Digital FIR filter involves 1-D convolution of filter coefficients and input data. Assume filter order $N=3$, POT digit limit to 2^{-4} .

$$y(n) = \sum_{k=0}^{N=3} h(k)x(n-k)$$

$$= h(0)x(n-0) + h(1)x(n-1) + h(2)x(n-2) + h(3)x(n-3)$$

Assume:

$$h(0) = 2^{-1} - 2^{-3}$$

$$h(1) = 2^{-3} + 2^{-4}$$

$$h(2) = 2^{-1} + 2^{-2} - 2^{-4}$$

$$h(3) = -2^{-3}$$

Traditional convolution equation that you implemented in HW#2 is:

$$y(n) = (2^{-1} - 2^{-3})x(n-0) + (2^{-3} + 2^{-4})x(n-1) + (2^{-1} + 2^{-2} - 2^{-4})x(n-2) + (-2^{-3})x(n-3)$$

But if you check section 3 of MSDAP paper, there is a more efficient way to do this convolution:

$$y(n) = 2^{-1}[x(n-0) + x(n-2)] + 2^{-2}[x(n-2)] + 2^{-3}[-x(n-0) + x(n-1) - x(n-3)] + 2^{-4}[x(n-1) - x(n-2)]$$

If we define::

$$u_4 = x(n - 0) + x(n - 2)$$

$$u_3 = x(n - 2)$$

$$u_2 = -x(n - 0) + x(n - 1) - x(n - 3)$$

$$u_1 = x(n - 1) - x(n - 2)$$

$$y(n) = 2^{-1}u_4 + 2^{-2}u_3 + 2^{-3}u_2 + 2^{-4}u_1$$

$$y(n) = 2^{-1}(u_4 + 2^{-1}(u_3 + 2^{-1}(u_2 + 2^{-1}u_1)))$$

Below is a general format for 16-bit shifting. This equation is what you will implement in this HW.

$$y(n) = 2^{-1}(u_{16} + 2^{-1}(u_{15} + \dots 2^{-1}(u_2 + 2^{-1}u_1) \dots))$$

Coefficients format (in MSDAP_Coeff*.in file):

Note in above, u_i can be pre-determined by $h(k)$. So, in this homework we do not use the $h(k)$ as a fraction. Instead, coefficients are provided in the format which is more convenient for calculating u_i . Here is the description of coefficient "038":

038=

s	Address bits							
0	0	0	1	1	1	0	0	0
MSB	9 bits							LSB

- Only 9 bits are used in the coefficient.
- MSB is the sign bit. 0 means addition and 1 means subtraction
- Rest of the 8 bits are the index to $x(n)$. In this case, it is $x(n-56)$. ($38_{\text{hex}} = 56_{\text{dec}}$)

Another example of a coefficient would be "18C"

18C=

s	address bit							
1	1	0	0	0	1	1	0	0
MSB	9 bits							LSB

- This means to subtract $x(n-140)$. Sign bit is 1 for subtraction. The rest of the bits are 140 in decimal.

R_j format (in R_j .in file)

In addition to coefficients, we need another parameter R_j to tell us the total number of terms when calculating u_j .

0B=

0	0	0	0	1	0	1	1
MSB	8 bits						LSB

- R_j is given by an 8-bit unsigned number, which tells the total number of terms for calculating u_j . For the above case, we need 11 addition/subtraction (determined by coefficients) operations when doing the computation.

Calculating u_j

When aligning coefficients and R_j , we can easily calculate u_j . Given coefficients and as follows:

038	18C	00F	096	16A	130	110	013	14F	1F1	0FA	08E	1FC	1B0	13E	0CF
R ₁ (0B)											R ₂ (0F)				
0EE	021	1AF	04C	0A5	076	0EB	13E	0A8	055	17A	078	0B4	03D	0B7	1DD
R ₂ (0F)										R ₃ (0E)					
169	16C	1F3	0E1	0B2	14E	1D3	0F8	14D	1FF	170	002	04A	1B0	17E	115
R ₃ (0E)								R ₄ (0D)							
132	1F9	15D	024	104	1A3	1BC	007	01A	1E4	0C3	095	0AF	1F7	08C	1C6
R ₄ (0D)					R ₅ (08)								R ₆ (0D)		
19C	0FC	00C	0FB	134	1E9	1AA	176	01C	075	0CC	14A	1B1	142	1B6	1B8

R ₆ (0D)										R ₇ (0D)					
1BB	09F	1FC	027	134	0D1	00F	189	130	199	04B	017	081	0E1	09D	10C
R ₇ (0D)							R ₈ (06)						R ₉ (05)		
1F3	02B	1D3	09C	1D8	01D	0A6	02D	179	0AE	122	076	012	094	090	141
R ₉ (05)		R ₁₀ (06)						R ₁₁ (04)				R ₁₂ (06)			
13D	104	062	0C1	093	068	132	183	0B6	0DD	0BD	1FE	0DD	07C	0E6	105
R ₁₂ (06)		R ₁₃ (09)									R ₁₄ (0B)				
1A2	0F1	00A	049	0AD	095	011	0E0	17C	1E3	1C2	0A7	1F8	12C	123	0C1
R ₁₄ (0B)						R ₁₅ (14)									
050	176	057	102	155	13F	1FD	1E7	159	047	018	13D	1E6	053	192	
R ₁₅ (14)										R ₁₆ (05)					

- Firstly, as $R_0 = 0B_{\text{hex}}$, we know the u_1 contains 11 terms.
- Secondly, as $\text{Coeff}(0) = 038B$, we know the first term of u_1 is $+x(n-56)$. And in the same way we can find all required terms.
- Thirdly, we look at R_2 and so on until all 16 u_i are calculated.

Take $x(2) = C48B$, $x(1) = 0$, $x(0) = 0$, as the first three inputs, assume $x(n) = 0$, $\forall n < 0$.

C48B: 1100 0100 1000 1011 // $x(2)$

ext. to 24-bit: 1111 1111 1100 0100 1000 1011 // $x(2)$

two's complement: 0000 0000 0011 1011 0111 0101 // $-x(2)$

The step-by-step computing procedure for $y(2)$ would be:

1 $u_1 = 0000 \quad 0000 \quad 0000 \quad 0000 \quad 0000 \quad 0000$

2 $2^{-1}u_1 = 0000 \quad 0000 \quad 0000 \quad 0000 \quad 0000 \quad 0000 \quad 0 \quad // \text{shifting then add}$

3 $(2+u_2) = 0000 \quad 0000 \quad 0000 \quad 0000 \quad 0000 \quad 0000 \quad 0$

4 $2^{-1}(2+u_2) = 0000 \quad 0000 \quad 0000 \quad 0000 \quad 0000 \quad 0000 \quad 00$

$$5 \quad (4+u_3) = 0000 \quad 0000 \quad 0000 \quad 0000 \quad 0000 \quad 0000 \quad 00$$

$$6 \quad 2^{-1}(4+u_3) = 0000 \quad 0000 \quad 0000 \quad 0000 \quad 0000 \quad 0000 \quad 000 \text{ // till now, you can easily verify } u_1 \sim u_3 \text{ are all 0s}$$

$$7 \quad (6+u_4) = \begin{array}{ccccccc} 0000 & 0000 & 0000 & 0000 & 0000 & 0000 & 000 \\ +1111 & 1111 & 1100 & 0100 & 1000 & 1011 & 000 \\ \hline =1111 & 1111 & 1100 & 0100 & 1000 & 1011 & 000 \end{array}$$

// due to "002" coefficient (see 1st red block in above figure)

$$8 \quad 2^{-1}(6+u_4) = 1111 \quad 1111 \quad 1110 \quad 0010 \quad 0100 \quad 0101 \quad 1000$$

$$9 \quad (8+u_5) = 1111 \quad 1111 \quad 1110 \quad 0010 \quad 0100 \quad 0101 \quad 1000$$

$$10 \quad 2^{-1}(8+u_5) = \quad 1111 \quad 1111 \quad 1111 \quad 0001 \quad 0010 \quad 0010 \quad 1100 \quad 0$$

$$11 \quad (10+u_6) = \quad 1111 \quad 1111 \quad 1111 \quad 0001 \quad 0010 \quad 0010 \quad 1100 \quad 0$$

$$12 \quad 2^{-1}(10+u_6) = \quad 1111 \quad 1111 \quad 1111 \quad 1000 \quad 1001 \quad 0001 \quad 0110 \quad 00$$

$$13 \quad (12+u_7) = \quad 1111 \quad 1111 \quad 1111 \quad 1000 \quad 1001 \quad 0001 \quad 0110 \quad 00$$

$$14 \quad 2^{-1}(12+u_7) = \quad 1111 \quad 1111 \quad 1111 \quad 1100 \quad 0100 \quad 1000 \quad 1011 \quad 000$$

$$15 \quad (14+u_8) = \quad 1111 \quad 1111 \quad 1111 \quad 1100 \quad 0100 \quad 1000 \quad 1011 \quad 000$$

$$16 \quad 2^{-1}(14+u_8) = \quad 1111 \quad 1111 \quad 1111 \quad 1110 \quad 0010 \quad 0100 \quad 0101 \quad 1000$$

$$17 \quad (16+u_9) = \quad 1111 \quad 1111 \quad 1111 \quad 1110 \quad 0010 \quad 0100 \quad 0101 \quad 1000$$

$$18 \quad 2^{-1}(16+u_9) = \quad 1111 \quad 1111 \quad 1111 \quad 1111 \quad 0001 \quad 0010 \quad 0010 \quad 1100 \quad 0$$

$$19 \quad (18+u_{10}) = \quad 1111 \quad 1111 \quad 1111 \quad 1111 \quad 0001 \quad 0010 \quad 0010 \quad 1100 \quad 0$$

$$20 \quad 2^{-1}(18+u_{10}) = 1111 \quad 1111 \quad 1111 \quad 1111 \quad 1000 \quad 1001 \quad 0001 \quad 0110 \quad 00$$

$$21 \quad (20+u_{11}) = 1111 \quad 1111 \quad 1111 \quad 1111 \quad 1000 \quad 1001 \quad 0001 \quad 0110 \quad 00$$

$$22 \quad 2^{-1}(20+u_{11}) = 1111 \quad 1111 \quad 1111 \quad 1111 \quad 1100 \quad 0100 \quad 1000 \quad 1011 \quad 000$$

$$23 \quad (22+u_{12}) = 1111 \quad 1111 \quad 1111 \quad 1111 \quad 1100 \quad 0100 \quad 1000 \quad 1011 \quad 000$$

$$24 \quad 2^{-1}(22+u_{12}) = 1111 \quad 1111 \quad 1111 \quad 1111 \quad 1110 \quad 0010 \quad 0100 \quad 0101 \quad 1000$$

$$25 \quad (24 + u_{13}) = 1111 \ 1111 \ 1111 \ 1111 \ 1110 \ 0010 \ 0100 \ 0101 \ 1000$$

$$26 \quad 2^{-1}(24 + u_{13}) = 1111 \ 1111 \ 1111 \ 1111 \ 1111 \ 0001 \ 0010 \ 0010 \ 1100 \ 0$$

$$27 \quad (26 + u_{14}) = 1111 \ 1111 \ 1111 \ 1111 \ 1111 \ 0001 \ 0010 \ 0010 \ 1100 \ 0$$

$$28 \quad 2^{-1}(26 + u_{14}) = 1111 \ 1111 \ 1111 \ 1111 \ 1111 \ 1000 \ 1001 \ 0001 \ 0110 \ 00 \ //$$

you can verify $u_5 \sim u_{14}$ are all 0s.

$$29 \quad (28 + u_{15}) = \begin{array}{r} 1111 \ 1111 \ 1111 \ 1111 \ 1111 \ 1000 \ 1001 \ 0001 \ 0110 \ 00 \\ +0000 \ 0000 \ 0011 \ 1011 \ 0111 \ 0101 \ 0000 \ 0000 \ 0000 \ 00 \\ \hline 0000 \ 0000 \ 0011 \ 1011 \ 0110 \ 1101 \ 1001 \ 0001 \ 0110 \ 00 \end{array}$$

// due to "102" coefficient (see 2nd red block in above figure) add $x(0)$

$$30 \quad 2^{-1}(28 + u_{15}) = 0000 \ 0000 \ 0001 \ 1101 \ 1011 \ 0110 \ 1100 \ 1000 \ 1011 \ 000$$

$$31 \quad (30 + u_{16}) = 0000 \ 0000 \ 0001 \ 1101 \ 1011 \ 0110 \ 1100 \ 1000 \ 1011 \ 000$$

$$32 \quad 2^{-1}(30 + u_{16}) = 0000 \ 0000 \ 0000 \ 1110 \ 1101 \ 1011 \ 0110 \ 0100 \ 0101 \ 1000$$

Hex= 0 0 0 E D B 6 4 5 8

$$y(2) = 000EDB6458$$

Homework requirements

(1) Write a well-organized ASIC specification for MSDAP.

The Specification should at least cover the following topics.

(1.1) Description of MSDAP algorithm. Give an example of convolution. (Note: Do not use the example in homework)

(1.2) Pin settings for MSDAP Chip

(1.3) Operation modes (descriptions of FSM)

(1.4) Input/Output signal formats

(1.5) The frequency of "SCLK" and how you determine it.

(2) Implement the reference model of the MSDAP in C/C++/Python

(2.1) Implement above convolution procedure with only plus and shift operations. (multiplication and subtraction are not allowed, use 2's complement for subtraction).

(2.2) Your report should include a section explaining how you implemented the algorithm.

- Input samples ($x(n)$) are signed 16-bit hex number, fixed-point, two's complement, Leftmost bit is sign bit similar to HW4.
- The output data should be 40 bits and saved as hexadecimal number
- Assume $x(-255) = \dots = x(-2) = x(-1) = 0$.

(2.3) On eLearning: data1.in / MSDAP_Coeff1.in / Rj1.in and output1.out are for testing purpose

(2.4) Save your output data (for data2.in / MSDAP_Coeff2.in / Rj2.in) in hex number to the file named as data2.out.

(2.5) Modify your C/C++/Python code in HW#2 (convolution golden reference) to create one new set of test input and output reference vectors (data3.in / MSDAP_Coeff3.in / Rj3.in / data3_reference.out). Save the output data in hex number to data3.out.

- Provide a written explanation if data3.out is different from data3_reference.out and why

Notes:

- If using C/C++ use gcc/g++ to compile your code. You could test your code with gcc/g++ on server engnx.utdallas.edu. Compiling errors of your source code file (like copying code into .doc, .docx or .pdf) will lead your grade to be 0.
- Your program should accept files **MSDAP_coeff.in**, **rj.in**, **data.in**, **data.out** as arguments. For testing purposes, TA will call your program using the following format.

```
./msdap.o <path/to/MSDAP_coeff.in> <path/to/rj.in> <path/to/data.in> \
<path/to/data.out>
```

- If you program in Python, kindly provide a Jupyter notebook and any instructions to assist the TA to run your code.